

FACULDADE ENGENHEIRO SALVADOR ARENA

MODULADOR NRZ-L

Tutorial de simulação no Vivado

1) PRÉ-REQUISITOS

Para iniciar a simulação, é necessário que o software Vivado seja instalado, na versão 2025.2. Para obtê-lo, baixe o instalador por meio do [site oficial](#).

Para que o Vivado possa ser utilizado normalmente, recomenda-se que os seguintes requisitos sejam atendidos:

- Sistemas operacionais Windows 10 Professional/Enterprise ou Windows 11 Enterprise (64-bit)
- Ao menos 16GB de RAM;
- Ao menos 50GB de armazenamento livre em disco para download dos componentes e armazenamento dos projetos;

2) ABERTURA DO PROJETO

No repositório do github, acesse a Branch do projeto para realizar o download. Com a Branch localizada, clique em “Code” e em “Download ZIP”. Um arquivo será baixado no computador:

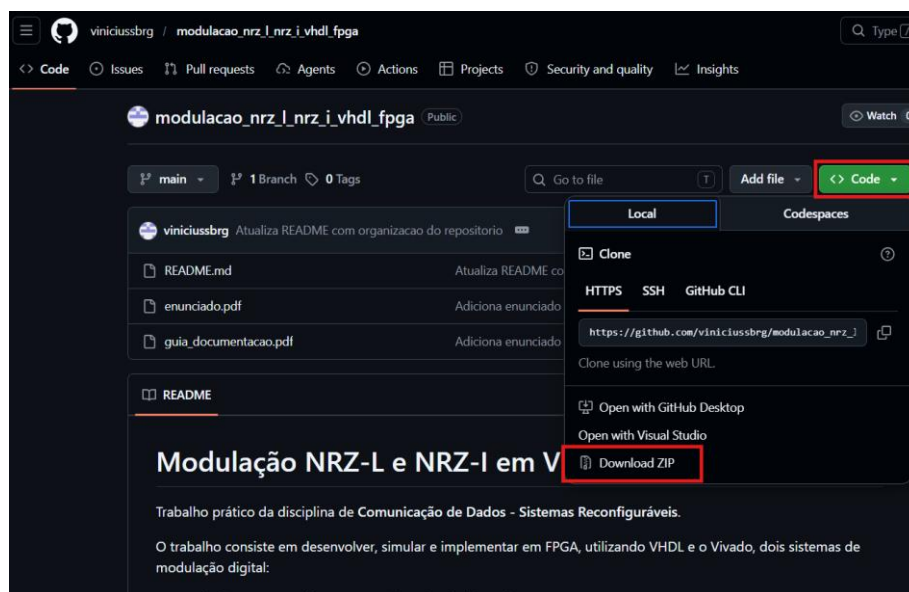


Figura 1 - Download do repositório no GitHub

Em seguida, extraia o arquivo em alguma pasta de sua preferência. Dentro do repositório do projeto já descompactado, navegue até o caminho “modulacao_nrz_l_nrz_i_vhdl_fpga\nrz_l\vivado_project” e extraia o arquivo ZIP “projeto_vivado_NRZ-L”, que está dentro deste diretório:

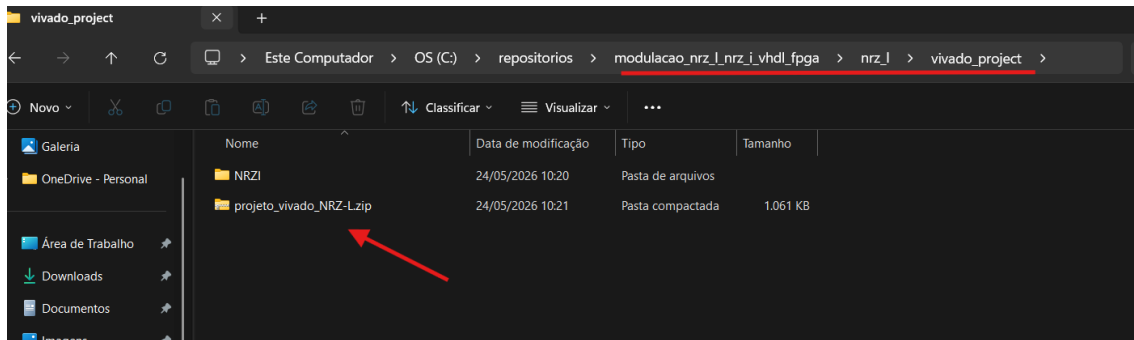


Figura 2 - Projeto descompactado

Abrindo o software Vivado, clique em “Open Project”. Será aberta uma janela para localizar o projeto em seu computador. Por meio do seletor “Look in”, navegue até o diretório no qual o projeto foi extraído. Entrando no projeto, localize um arquivo de extensão “.xpr”, e clique duas vezes:

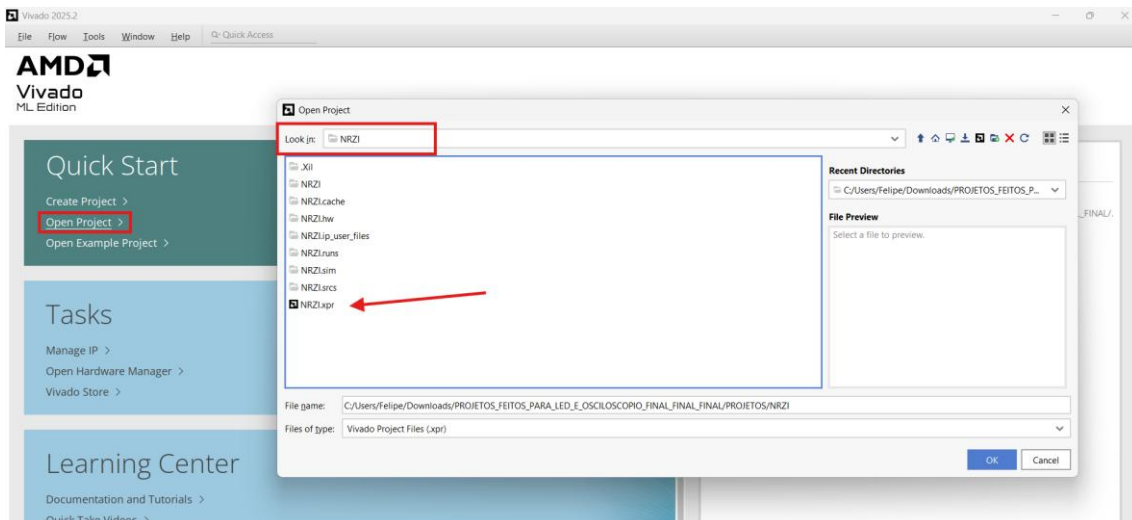


Figura 3 - Abertura do projeto no Vivado

Ao final, a página inicial do Vivado deve ser aberta:

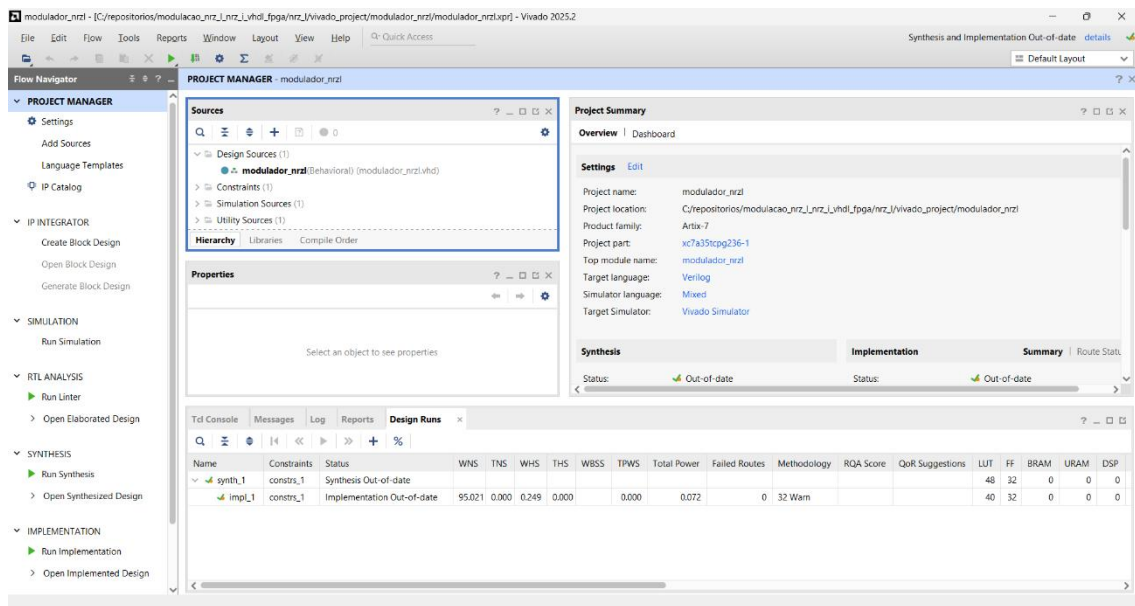


Figura 4 - Tela inicial do Vivado

3) ESTRUTURA DOS ARQUIVOS NO VIVADO

Dentro da tela inicial do Vivado, é possível visualizar na área Sources os arquivos de “Design Source”, “Constraints” e “Simulation Sources”:

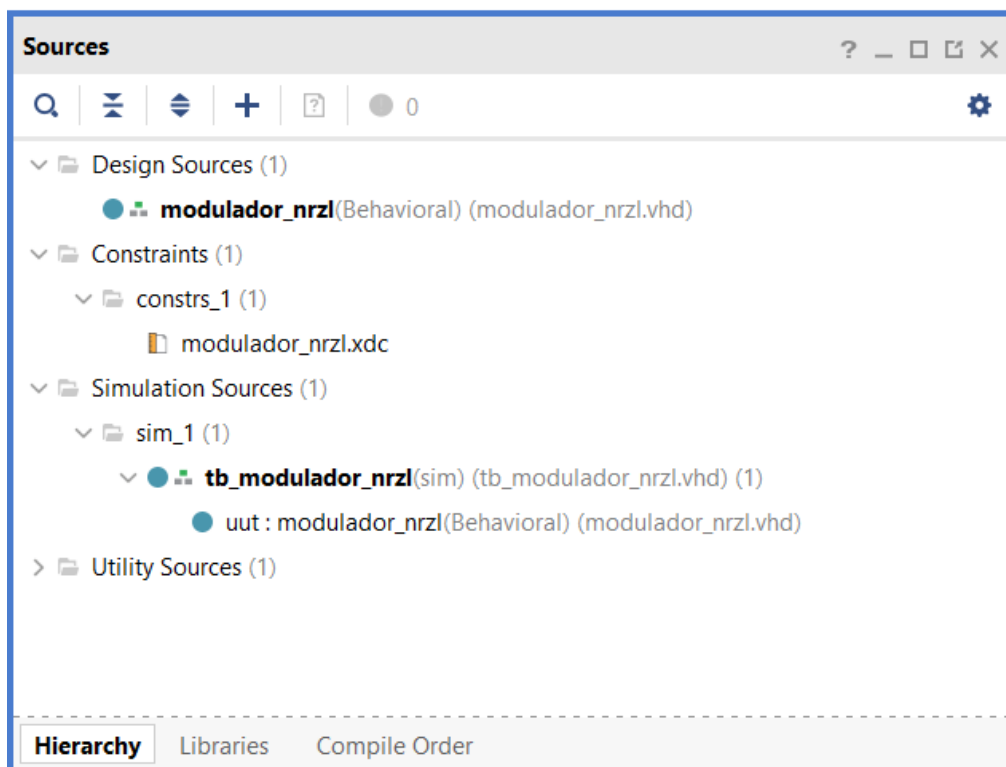


Figura 5 - Área Sources do Vivado

Na tabela a seguir, descreve-se qual a função de cada um destes arquivos:

Arquivo	Função
Design Source (modulador_nrzl)	Contém o código principal do circuito FPGA.
Constraints (modulador_nrzl.xdc)	Contém o mapeamento das entradas e saídas do circuito definidas no Design Source, para vinculá-las às portas do circuito físico.
Simulation Source (tb_modulador_nrzl)	Contém o código necessário para simular o funcionamento da placa FPGA no Vivado, sem precisar do hardware.

Tabela 1 - Arquivos do circuito no Vivado

Clicando duas vezes no arquivo do design source, o seu código-fonte será aberto. Mais detalhes sobre seu funcionamento podem ser vistos por meio dos comentários (linhas acinzentadas):

```

1 | library IEEE;
2 | use IEEE.STD_LOGIC_1164.ALL;
3 | use IEEE.NUMERIC_STD.ALL;
4 |
5 | entity modulador_nrzl is
6 |     generic (
7 |         NUM_BITS: integer := 16; -- número de bits a serem transmitidos
8 |         CICLOS_FOR_BIT: integer := 2000 -- número de ciclos de clock por bit
9 |     );
10 | port (
11 |     clock: in std_logic; -- porta que representa o sinal de clock
12 |     entrada: in std_logic_vector(0 to NUM_BITS-1); -- input do testbench ou dos switches da placa física
13 |     saida_testbench: out std_logic; -- output no waveform (via testbench)
14 |     saida_leds: out std_logic_vector(0 to NUM_BITS-1) -- output nos leds da placa física
15 | );
16 | end entity;
17 |
18 |
19 | architecture Behavioral of modulador_nrzl is
20 |     -- indicação do bit atual sendo transmitido.
21 |     signal bit_index : integer range 0 to NUM_BITS := 0;
22 |
23 |     -- contador que controle a duração de cada bit em ciclos de clock.
24 |     signal contador_clock : integer range 0 to CICLOS_FOR_BIT := 0;
25 |
26 |     -- registro para armazenar o estado dos LEDs, inicializado com '0' (todos apagados).
27 |     signal leds_reg: std_logic_vector(0 to NUM_BITS-1) := (others => '0');
28 | begin
29 |     -- linkagem do registro de LEDs à saída dos LEDs da placa física.
30 |     saida_leds <= leds_reg;
31 |
32 |     process(clock)
33 |     begin
34 |         if rising_edge(clock) then -- se houver uma borda de subida no clock, o processo é executado
35 |             if bit_index < NUM_BITS then -- verifica se ainda há bits para transmitir
36 |                 saida_testbench <= entrada(bit_index); -- modifica sinal no waveform;
37 |                 leds_reg(bit_index) <= entrada(bit_index); -- acende/apaga o led respectivo ao bit;
38 |
39 |                 if contador_clock < CICLOS_FOR_BIT - 1 then -- verifica se ainda não terminou o período do bit atual.
40 |                     contador_clock <= contador_clock + 1;
41 |                 else -- se terminou, reinicia o contador e passa para o próximo bit.
42 |                     contador_clock <= 0;
43 |                     bit_index <= bit_index + 1;
44 |                 end if;
45 |
46 |             else -- se a sequência foi transmitida, reinicia tudo para o próximo ciclo de transmissão.
47 |                 bit_index <= 0;
48 |                 contador_clock <= CICLOS_FOR_BIT - 1; -- põe o contador do clock no final para reiniciar pulso corretamente
49 |                 leds_reg <= (others => '0');
50 |             end if;
51 |
52 |         end if;
53 |     end process;
54 |
55 | end Behavioral;

```

Figura 6 - Código-fonte do Design Source

Clicando duas vezes no arquivo de constraints, o seu código-fonte será aberto. O que se faz aqui, conforme explicado acima, é indicar ao hardware físico

a relação de entradas e saídas que serão utilizadas. No arquivo completo, vê-se a área de definições para as entradas (Switches), saídas digitais (os LEDs) e o clock:

```

1  ## =====
2  ## Switches
3  ## index 15 (mais à esquerda) = bit mais significativo
4  ## index 0 (mais à direita) = bit menos significativo
5  ## =====
6
7  set_property PACKAGE_PIN V17 [get_ports {entrada[15]}]
8  set_property IOSTANDARD LVCN0533 [get_ports {entrada[15]}]
9
10 set_property PACKAGE_PIN V16 [get_ports {entrada[14]}]
11 set_property IOSTANDARD LVCN0533 [get_ports {entrada[14]}]
12
13 set_property PACKAGE_PIN W16 [get_ports {entrada[13]}]
14 set_property IOSTANDARD LVCN0533 [get_ports {entrada[13]}]
15
16 set_property PACKAGE_PIN W17 [get_ports {entrada[12]}]
17 set_property IOSTANDARD LVCN0533 [get_ports {entrada[12]}]
18
19 set_property PACKAGE_PIN W15 [get_ports {entrada[11]}]
20 set_property IOSTANDARD LVCN0533 [get_ports {entrada[11]}]
21
22 set_property PACKAGE_PIN V15 [get_ports {entrada[10]}]
23 set_property IOSTANDARD LVCN0533 [get_ports {entrada[10]}]
24
25 set_property PACKAGE_PIN W14 [get_ports {entrada[9]}]
26 set_property IOSTANDARD LVCN0533 [get_ports {entrada[9]}]
27
28 set_property PACKAGE_PIN W13 [get_ports {entrada[8]}]
29 set_property IOSTANDARD LVCN0533 [get_ports {entrada[8]}]
30
31 set_property PACKAGE_PIN V2 [get_ports {entrada[7]}]
32 set_property IOSTANDARD LVCN0533 [get_ports {entrada[7]}]

```

Figura 7 - Parte do código das constraints do circuito

Clicando duas vezes no arquivo do simulation source, o seu código-fonte será aberto. Mais detalhes sobre seu funcionamento podem ser vistos por meio dos comentários (linhas acinzentadas):

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity tb_modulador_nrsl is
5  end entity;
6
7  architecture sim of tb_modulador_nrsl is
8
9      constant NUM_BITS : integer := 16; -- número de bits a serem transmitidos
10     constant PERIODO_CLOCK : time := 100 ns; -- garante clock a 10MHz
11     constant PERIODO_BIT : time := 200 ns; -- define o período de cada bit para 200ns.
12     constant CICLOS_FOR_BIT_CALC : integer := PERIODO_BIT / PERIODO_CLOCK; -- calcula o número de ciclos de clock por bit com base no período definido.
13
14     signal clock : std_logic := '0'; -- sinal para emular o sinal de clock
15     signal saida_testbench : std_logic; -- sinal para observar a saída do testbench no waveform
16     signal entrada : std_logic_vector(0 to NUM_BITS - 1) := "1011001110001111"; -- vetor de bits a ser transmitido, pode ser modificado para testar diferentes sequências
17
18     begin
19
20         -- instanciando o módulo a ser testado, mapeando os sinais e parâmetros do testbench para as portas e generics do módulo.
21         uut: entity work.modulador_nrsl
22             generic map (
23                 NUM_BITS => NUM_BITS,
24                 CICLOS_FOR_BIT => CICLOS_FOR_BIT_CALC
25             )
26             port map (
27                 clock => clock,
28                 saida_testbench => saida_testbench,
29                 entrada => entrada
30             );
31
32         -- Definição do Clock
33         p_clk: process
34         begin
35             clock <= '0'; wait for PERIODO_CLOCK / 2;
36             clock <= '1'; wait for PERIODO_CLOCK / 2;
37         end process;
38     end architecture;
39

```

Figura 8 - Código-fonte do simulation Source do circuito

4) SIMULAÇÃO

Para realizar a simulação via testbench do Vivado, alguns ajustes podem ser necessários. Caso queira modificar o período de cada ciclo de clock, o período de cada bit (ambos em nanossegundos) ou o conjunto de bits de dados que serão transmitidos (sempre 16 bits), modifique as variáveis das linhas 10, 11 e 16 respectivamente, no arquivo do simulation source:

```
9:
10: constant NUM_BITS : Integer := 16; -- número de bits a serem transmitidos
11: constant PERIODO_CLOCK : time := 100 ns; -- garante clock a 10MHz
12: constant PERIODO_BIT : time := 200 ns; -- define o período de cada bit para 200ns.
13: constant CICLOS_FOR_BIT_CALC : Integer := PERIODO_BIT / PERIODO_CLOCK; -- calcula o número de ciclos de clock por bit com base no período definido.
14:
15: signal clock : std_logic := '0'; -- sinal para emular o sinal de clock
16: signal saída_testbench : std_logic; -- sinal para observar a saída do testbench no waveform
17: signal entrada : std_logic_vector(0 to NUM_BITS - 1) := "1011001110001111"; -- vetor de bits a ser transmitido, pode ser modificado para testar diferentes sequências
```

Figura 9 - Ajuste de parâmetros no simulation source

No menu Flow Navigator, Clique em Run Simulation → Run Behavioral Simulation:

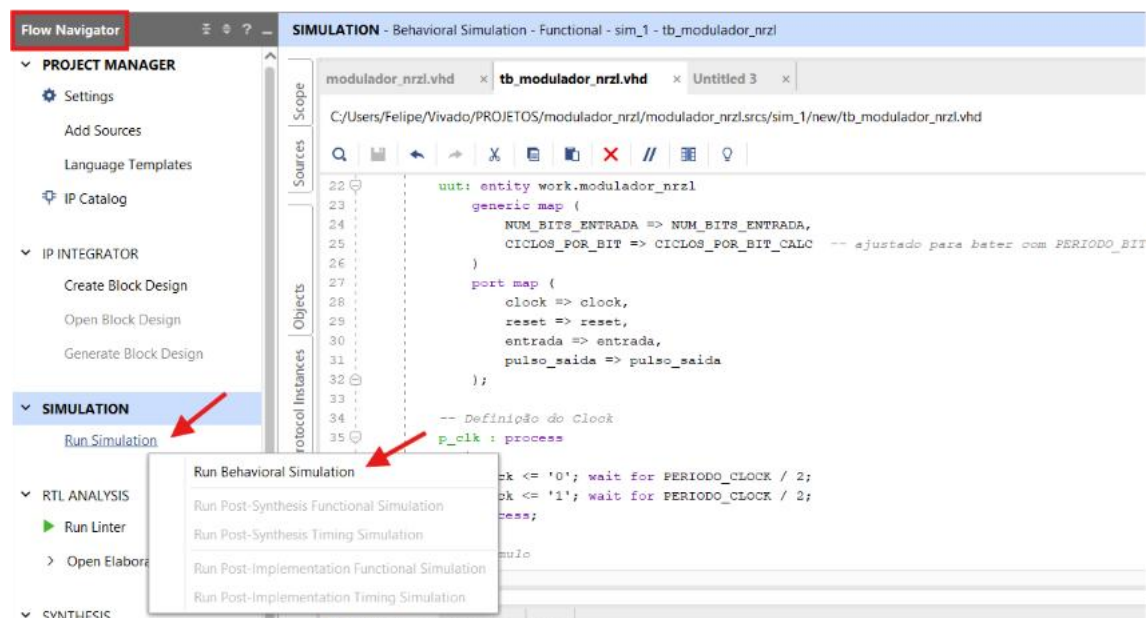


Figura 10 - Execução da simulação via testbench

Após isso, será aberta a tela do simulador. Para facilitar a visualização do pulso, seguindo a imagem abaixo, resete a simulação clicando em Restart (azul), mude a duração dela para 100 us (verde), inicie clicando em “Run for” (vermelho) e clique em Zoom Fit (roxo). Em seguida, clique em “Zoom In” (amarelo) o quanto for necessário, para se aproximar mais do pulso.

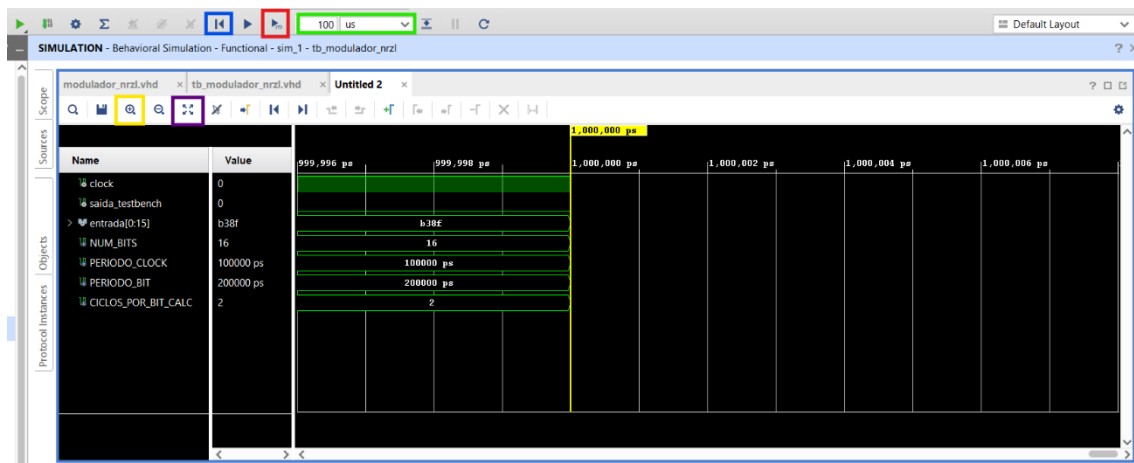


Figura 11 – Tela Waveform da simulação

Com a visualização ajustada, espera-se que o pulso tenha um formato análogo ao demonstrado abaixo:

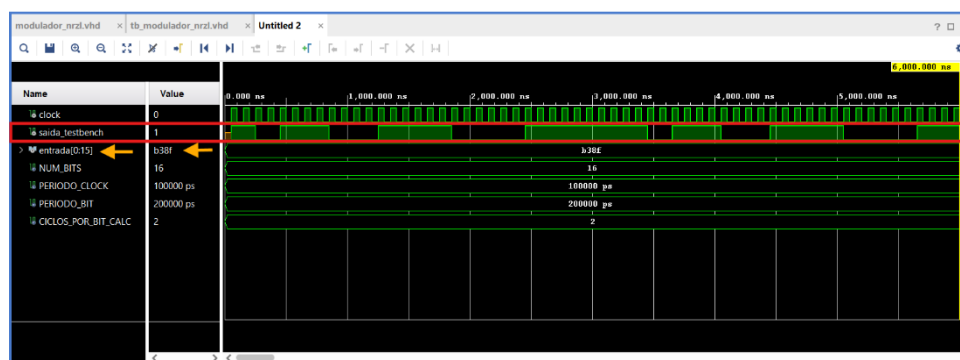


Figura 12 - Pulso da modulação no Waveform ajustado

Observa-se o pulso de saída (indicação vermelha), transmitindo do Bit mais significativo ao menos significativo e o conjunto de bits de entrada (setas laranjas). OBS: a string “b38f” do exemplo também pode ser lida como “1011001110001111”. A execução inicia assim que o clock atinge a borda de subida.

Com base no resultado obtido, vê-se que o pulso se comporta conforme a lógica prevista na modulação NRZ-L. De forma simples, os bits “1” deixam o nível do sinal alto e os bits “0” deixam o nível baixo.